

An introduction to (Big) Data Integration

TECHNOLOGIES FOR INFORMATION SYSTEMS

Letizia Tanca

Politecnico di Milano



The Data Integration problem is:

Combining data coming from different data sources,
providing the user with a unified vision of the data

→ Detecting correspondences between similar concepts that
come from different sources, and conflict solving

The four V's of Big data in data integration

- **Volume:** Not only can each data source contain a huge volume of data, but also the number of data sources has grown to be in the millions.
- **Velocity:** As a direct consequence of the rate at which data is being collected and continuously made available, many of the data sources are very dynamic.
- **Variety:** Data sources (even in the same domain) are extremely heterogeneous both at the schema level, regarding how they structure their data, and at the instance level, regarding how they describe the same real world entity, exhibiting considerable variety even for substantially similar entities.
- **Veracity:** Data sources (even in the same domain) are of widely differing qualities, with significant differences in the coverage, accuracy and timeliness of data provided. This is consistent with the observation that “1 in 3 business leaders do not trust the information they use to make decisions.”

Information Systems in the Era of Big data

The Variety dimension:

people and enterprises need to integrate data and the systems that handle those data: relational DBMSs and their extensions, legacy data and legacy DBMSs, sensors and user-generated content produce **heterogeneous**, structured or unstructured data

The Veracity dimension:

Data Quality is the most general and used term, and represents a number of quality aspects besides veracity:

- Completeness,
- Validity,
- Consistency,
- Timeliness
- Accuracy
- Ethics and fairness (a new entry!)

Heterogeneity derives from various forms of AUTONOMY

- Design (representation) autonomy,
- Communication (querying) autonomy, and
- Execution (algorithmic) autonomy

..... GENERATE HETEROGENEITY,

....while there is a need for interoperability among SW applications, services and information (databases, and others) managed by different organizations that need to reuse legacy applications, existing data repositories, and reconcile the different points of view adopted by the various players using the information.

**Data integration approaches interoperability
from a Data Perspective**

(Big) Data Integration Problems

VARIETY (heterogeneity) among several data collections to be used together

1. Different platforms: Technological heterogeneity
2. Different data models of the participating datasets → Model heterogeneity
3. Different query languages → Language heterogeneity
4. Different data schemas and different conceptual representations in DBs previously developed → Schema (semantic) heterogeneity
5. Different values for the same info (due to errors or to different knowledge) → Instance (semantic) heterogeneity

VERACITY: (Main) Data Quality dimensions:

1. Completeness,
2. Validity,
3. Consistency,
4. Timeliness
- 5.● Accuracy

The steps of Data Integration: how is it done?



Schema Reconciliation

Schema reconciliation: mapping the **data structure** (if it exists!)

Record Linkage

Record linkage (aka Entity resolution): data matching based on **the same content**

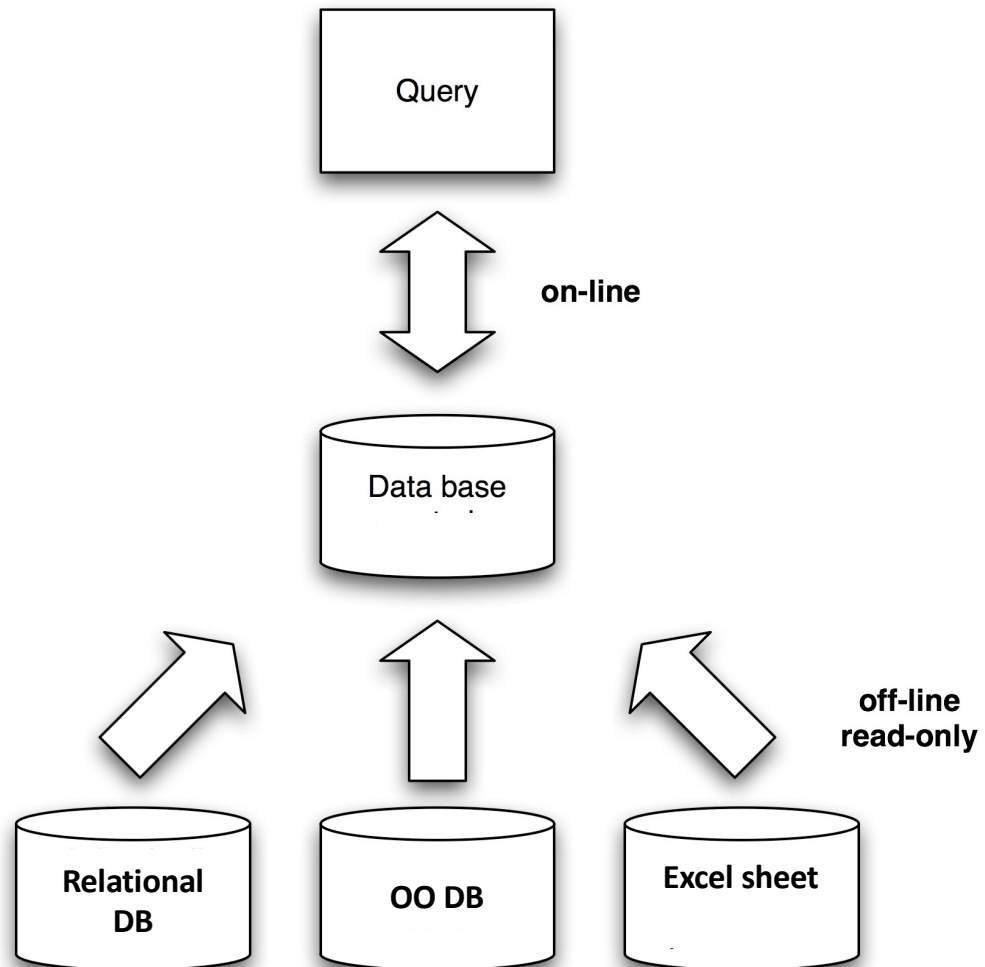
Data Fusion

Data fusion: reconciliation of **non-identical content**

Relevant Ways of Integrating Database Systems

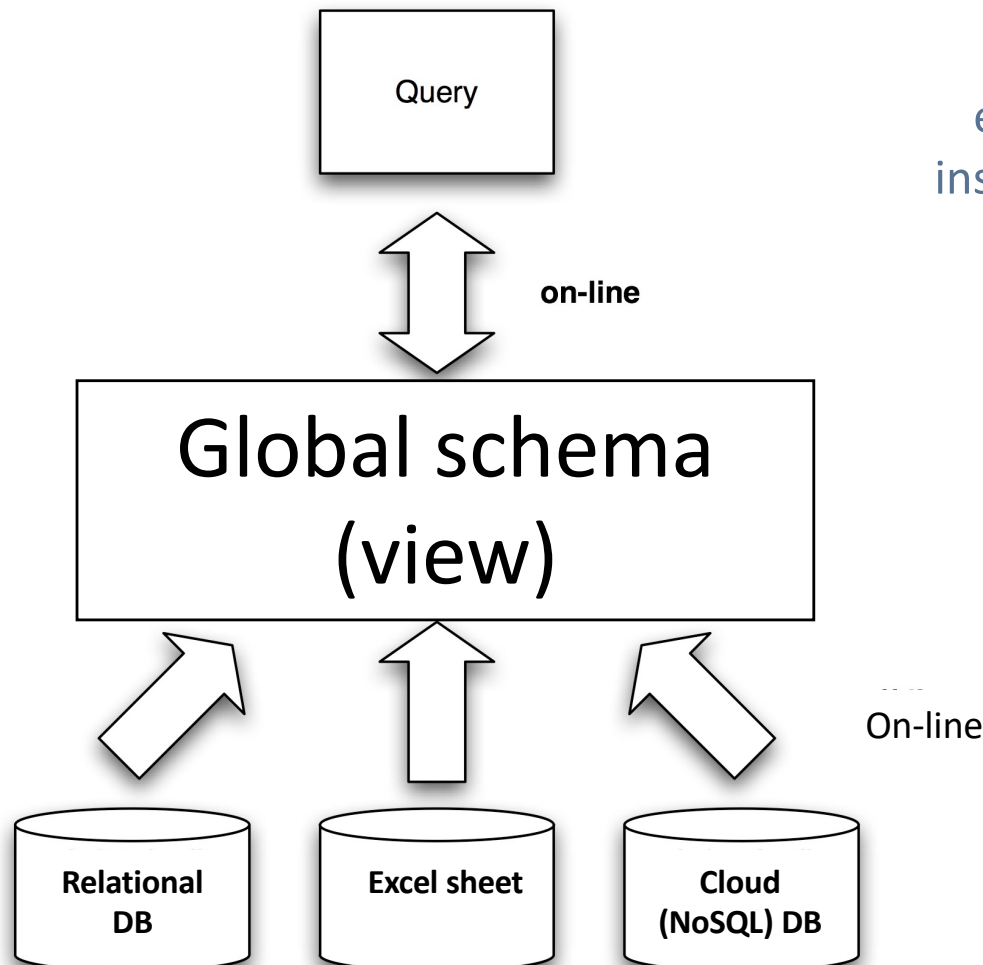
1. Use a **materialized database** (data are merged in a new database) → Extract-Transform-Load Systems
→ Data Warehouses: Materialized integrated data sources
2. Use a **virtual non-materialized data base** (data remain at sources) →
 - Enterprise Information Integration (EII) (or Data Integration) Systems (common front-end to the various datasources)
 - Data Exchange (source-to-target), less used recently

Materialized



Virtual

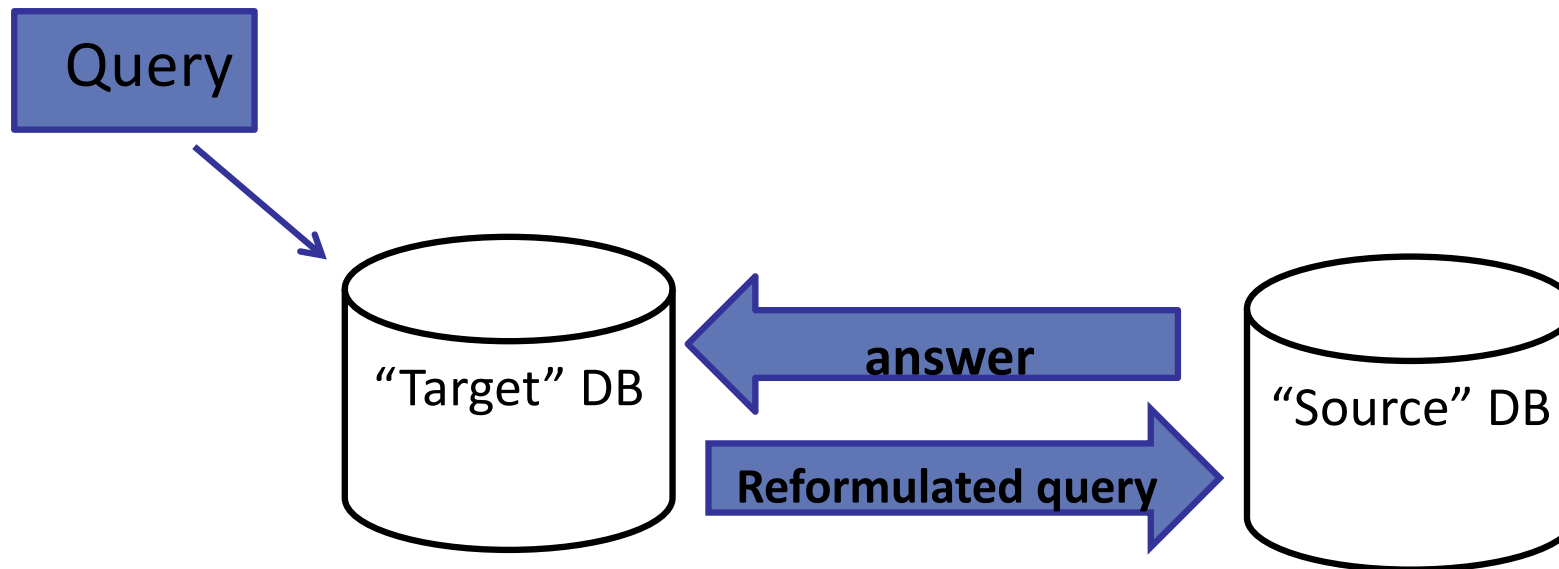
Given a target schema T , a set S of source schemas, a set M of mappings relating T to each element in S , and a query $Q(T)$ against the target schema, use M to evaluate $Q(T)$ over the set $I(S)$ of source instances (i.e., over the data stored in the sources).



Data Exchange:

virtual, between two sources e.g. IBM CLIO
(nowadays out-of-fashion)

Given a target schema T , a source schema S , a set of M mappings relating T and S , materialize the target instance $I(T)$ using the source instance $I(S)$ (i.e., the data stored currently stored in the source)



Sept 17th

Materialized integration

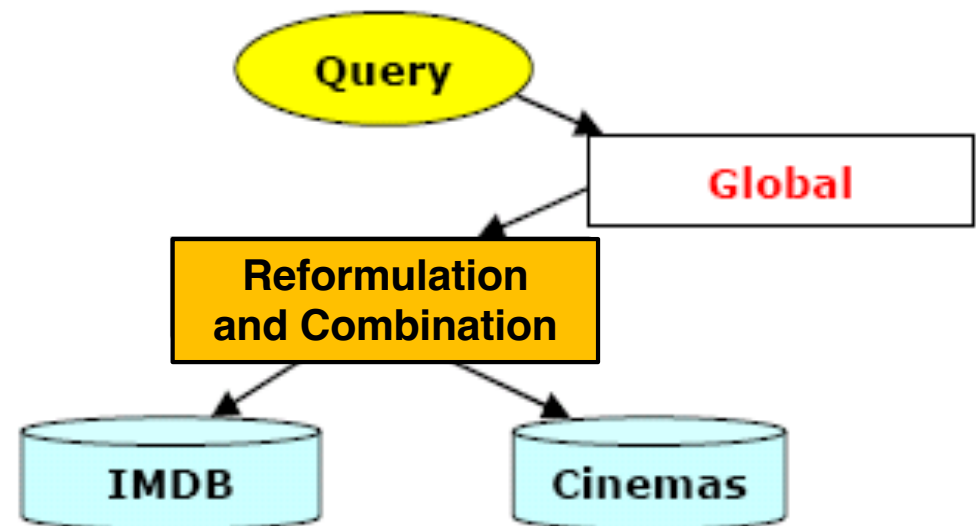
- Mostly used in data warehouses, special data collections that allow systematic or ad-hoc data analysis and mining, typically for enterprise strategical decision-making.
- The software used to fill a Data Warehouse is called an (ETL) system: to **Access**, **scrape**, **Transform**, and **Load** the data into the warehouse
- In a dynamic environment, ETL must be performed periodically (say once a day or week or month), thereby building up a history of the enterprise.
- Such a solution is not appropriate when the need is to integrate the operational systems (keeping the data continuously up-to-date)

Virtual integration

The virtual integration approach leaves the requested information in the local sources, therefore the virtual approach **will always return a fresh answer to the query**:

The query posted to the global schema is reformulated into the formats of the local information system.

The information retrieved needs to be combined to answer the query.



Rationale

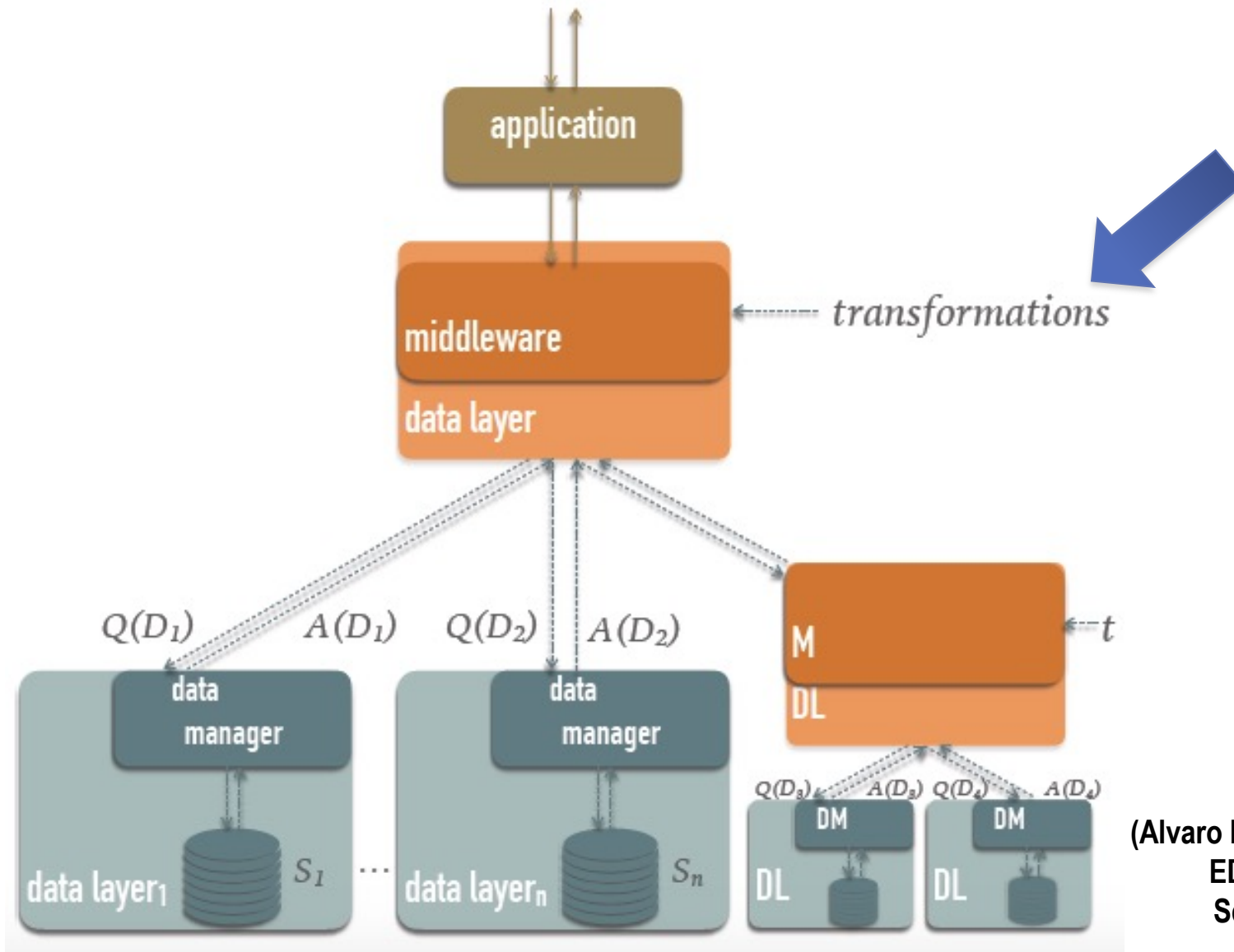
- The conventional wisdom is to use data warehousing and ETL products to perform data integration. However, there is a serious flaw in one aspect of this wisdom.
- Suppose one wants to integrate current (operational) data rather than historical information. Consider, for example, an e-commerce web site which wishes to sell hotel rooms over the web. The actual inventory of available hotel rooms exists in 100 or so information systems since Hilton, Hyatt and Marriott all run their own reservation systems.
- Applying ETL and warehousing to this problem will create a copy of hotel availability data which is **quickly out of date and, if a web site sells a hotel room based on this data, it has no way of guaranteeing delivery of the room, because somebody else may have sold the room in the meantime.**

HENCE:

when we need the integrated data to be always up-to-date,
WE MUST USE VIRTUAL INTEGRATION !!!



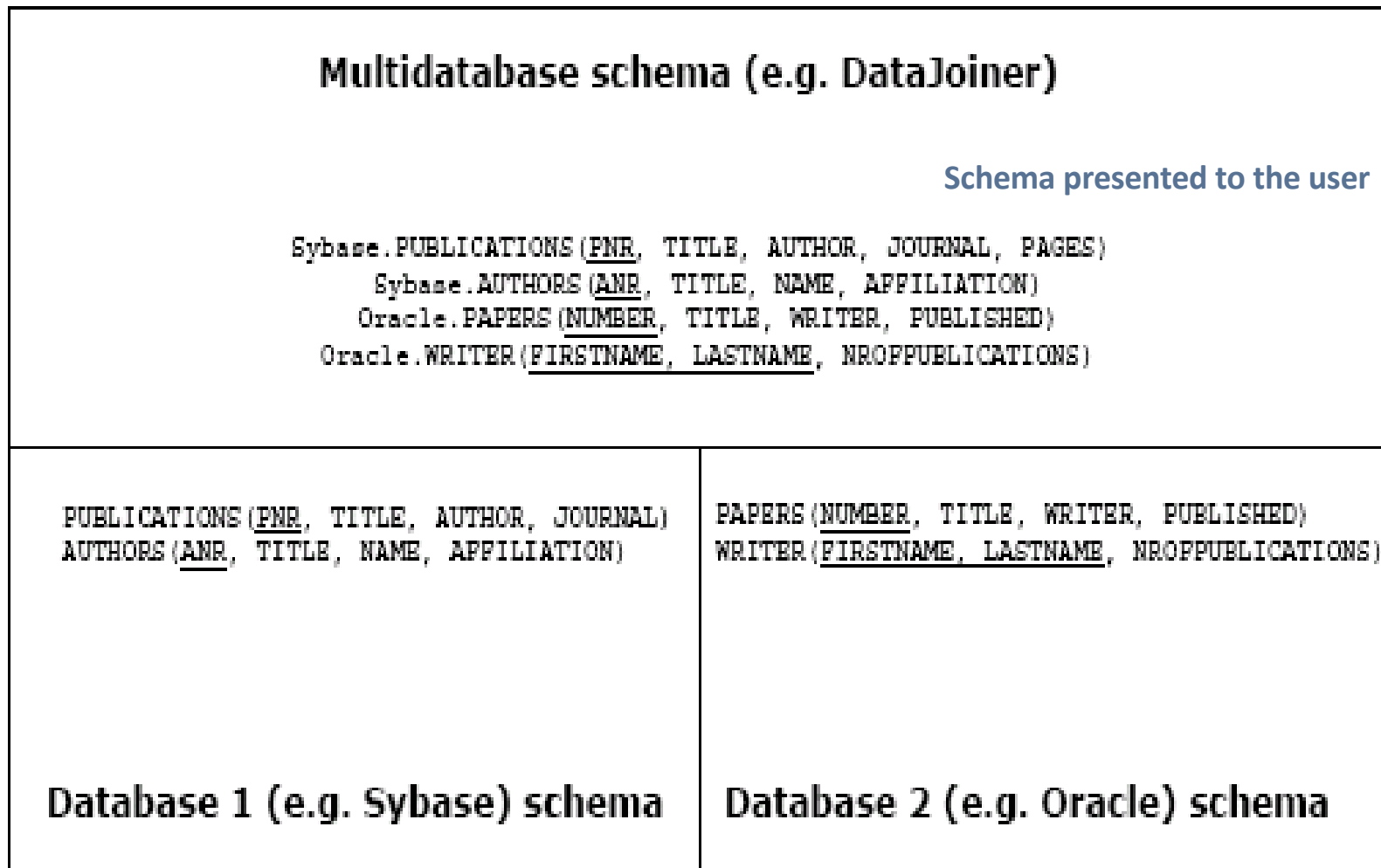
A general framework for Data Integration



(Alvaro Fernandez,
EDBT Summ.
School 2017)

A simple example (Batini)

In this very simple example, we don't have a "transformation" definition because the user sees directly the tables of the sources, therefore the transformations are "the identity mapping"



Query execution in the integration system

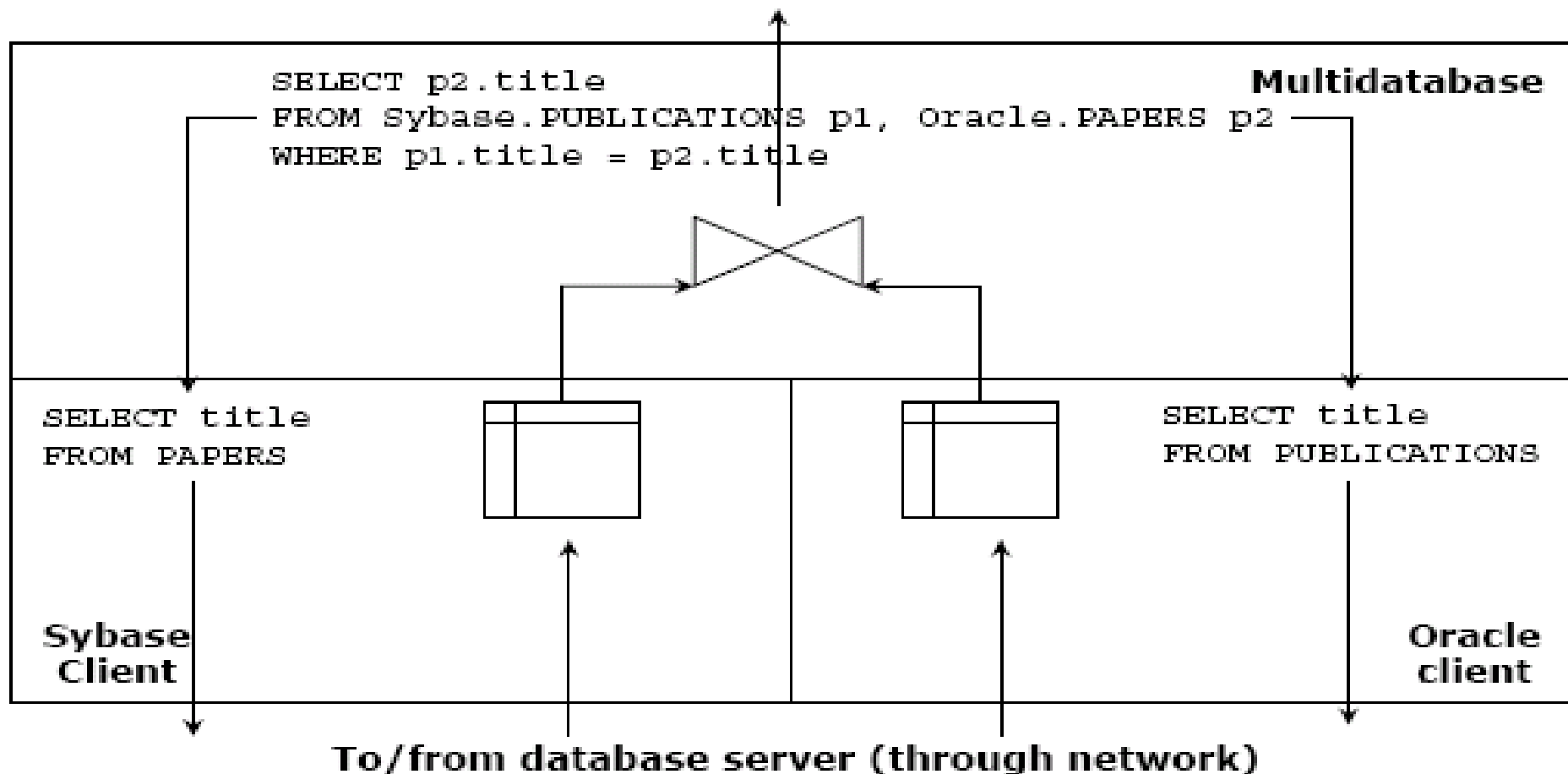
When a query is submitted, the integration system has to decompose it into queries against the component datasets.

1. determine first which parts of the query refer to which dataset:
 - which query parts refer to data from a single dataset and
 - which query parts refer to data from different datasets.

The latter ones can only be evaluated over the integrated data (view), whereas the former ones can be evaluated within the component datasets.

Example of query execution: a join

- The join needs to be evaluated at a global level.
- One of the main challenges in developing virtual data integration systems is to find good strategies of how to decompose and evaluate queries against multiple databases in an efficient manner.



The possible integration situations

Data integration problems arise even in the simplest situation: **when we have to design a unique**, centralized DB putting together a set of existing DBs independently designed...

...and integration becomes more and more complex...

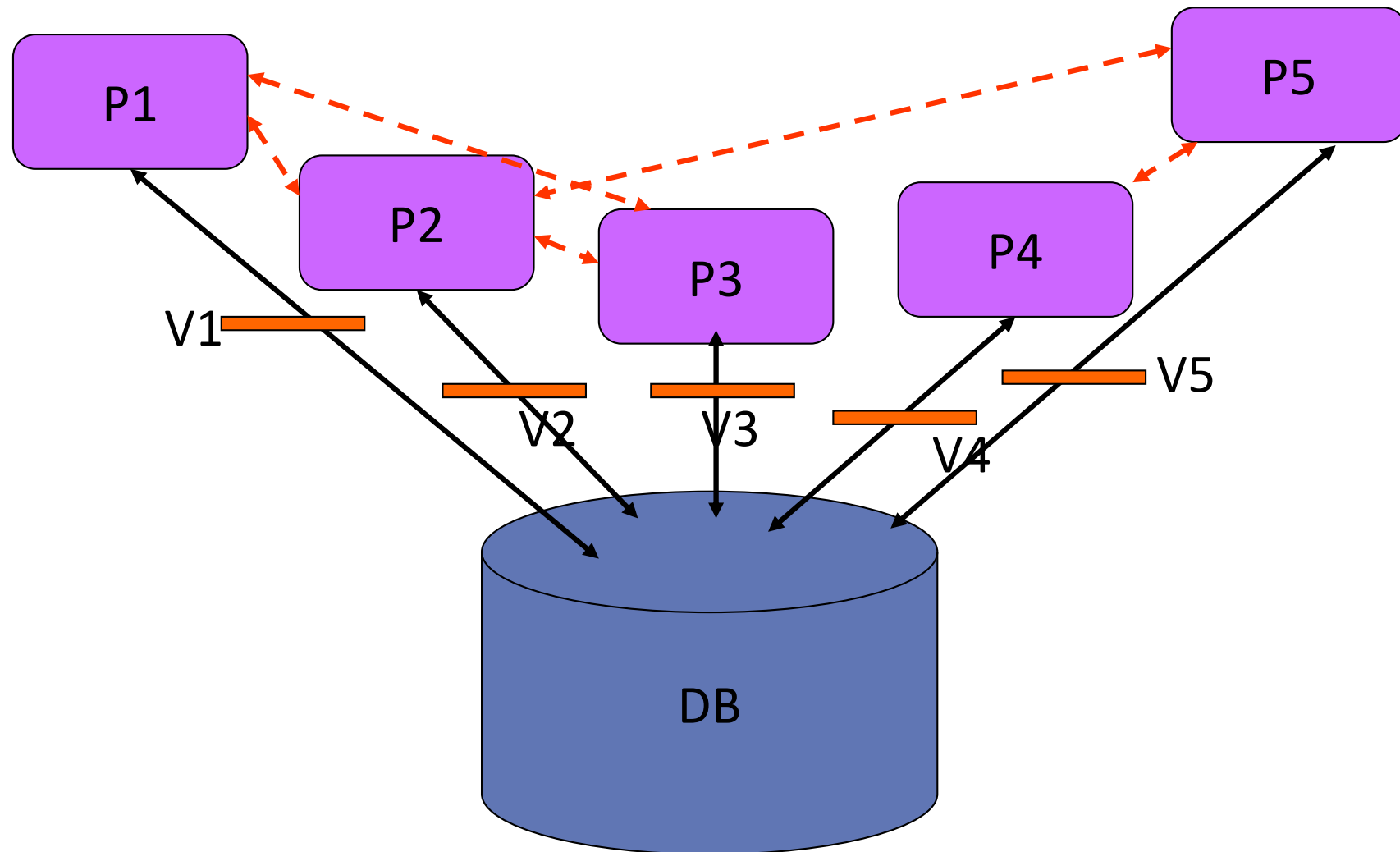
...up to the extreme case of **transient, dynamic, initially unknown** data sources...

Why should we need data integration in a UNIQUE DB ?

- A company has many departments: PURCHASES, PRODUCTION, LOGISTICS, PERSONNEL,.....
- Each area of the company interacts with the DB designer(s) for their (part of) DB
- However, a lot of this data are common to some of the areas (e.g., PURCHASES and PRODUCTION, or PRODUCTION and LOGISTICS)
- If the global company DB is just the collection of these partial DBs, there will be many redundancies, causing:
 - useless memory occupation
 - worst of all, when updating one instance of these duplicates, maybe the other one will remain as before (inconsistencies)

The correct answer:

within a single company, an integrated, centralized DB



Why data integration in a UNIQUE DB ?

So that each datum, whatever application uses it, **only appears once**. This **ELIMINATES** useless **REDUNDANCIES** which would cause:

- Inconsistencies (i.e. contradictions)
- Useless memory occupation

Each functionality in the company will have its own **specialized view**

This is achieved by using **view definitions** over the common DB

Views allow **personalization** as well as **access control**

Here and later we will use the concept of VIEW:
a conceptual tool to express the transformations

- Example of a view:

```
COURSES (CourseName, Teacher, RoomName)  
ROOMS (RoomName, BuildingNo, CampusName)
```

Create view *CourseCampus* as

Select *CourseName, CampusName*

From *Courses, Rooms*

Where *Courses.RoomName=Rooms.RoomName*

- View schema:

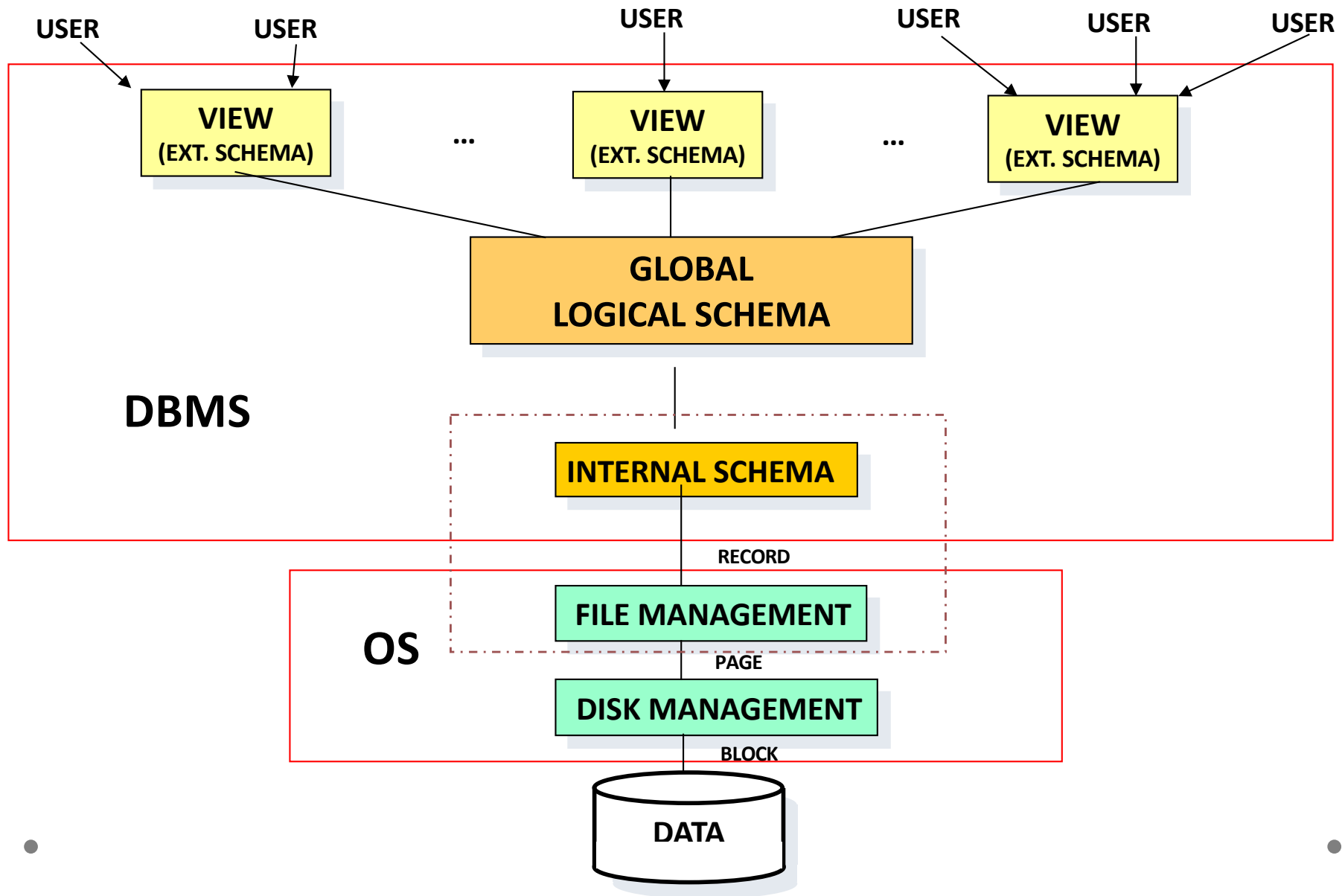
```
CourseCampus (CourseName, CampusName)
```

Query processing in a centralized DB

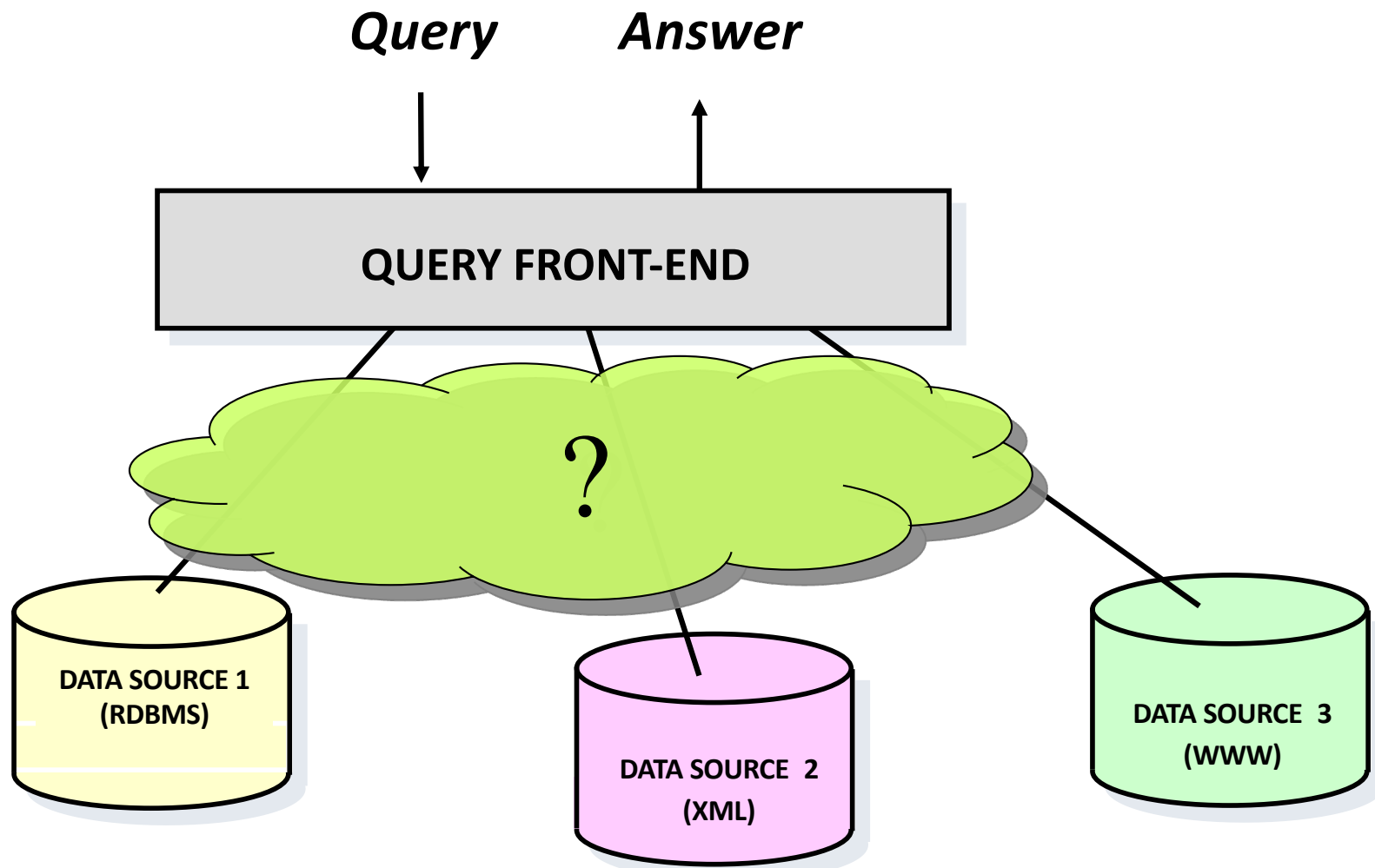
- User issues query Q against the user view $Q(V_i)$
- Query composition $\rightarrow Q \circ V_i(DB)$
- Answer is in terms of the base relations (global schema) or of the viewed relations (external schema), depending on how sophisticated the system is

Query processing in a centralized DB

(the ANSI/SPARC architecture)



Let us now reason on multiple data sources, concentrating first on Relational Databases



In general, the Data Integration problem is (recall):

Combining data coming from different data sources, providing the user with a unified vision of the data

→ Detecting correspondences between similar concepts that come from different sources, and conflict solving

Do not confuse what we do with Distributed DBs (*)

- It is the simplest case of non-centralized DB
- Often, data for the same organization
- Integrated a-priori: same design pattern as in the centralized situation, with homogeneous technology and data model
- Once the distributed database has been designed as if it were centralized, at *distribution design time* the design decisions concern:
 - Fragmentation:
 - Vertical
 - Horizontal
 - Allocation
 - Replication

(*) NOT a matter of data integration, therefore not our concern here:
see DB2 course

Recall: the steps of Data Integration with multiple sources



Schema Reconciliation

(If the sources have a schema)
Schema reconciliation: mapping the **data structure**

Record Linkage

Record linkage (aka Entity resolution): data matching based on **the same content**

Data Fusion

Data fusion: reconciliation of **non-identical content**

Approaches (recall)

- **Materialization**: data are converted and **stored** into a unique system, e.g. a Data Warehouse.

Problems:

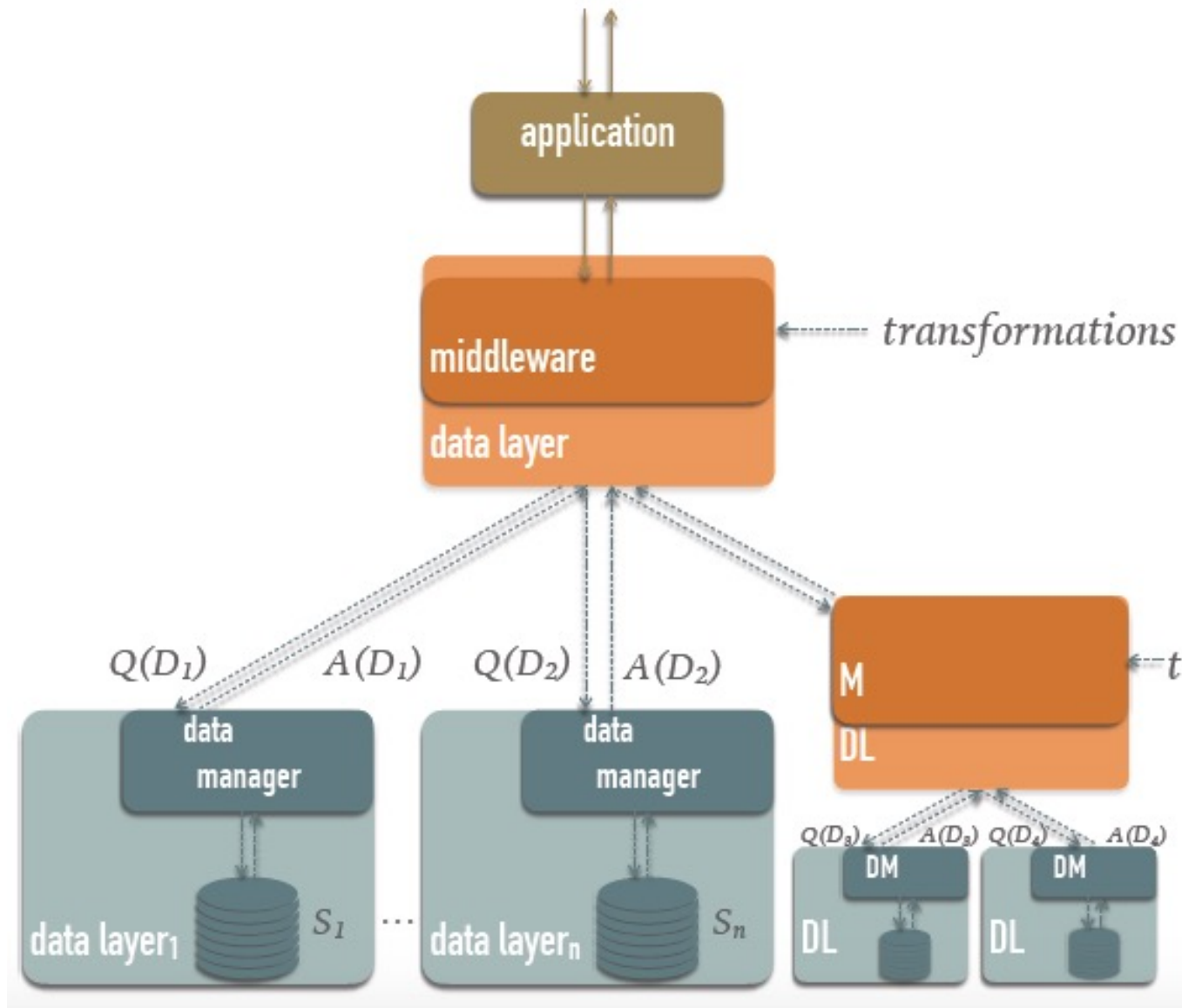
- Multiple copies: untimeliness, redundancy, inconsistency
 - Application rewriting if one system is discarded
-
- **Multidatabase (virtual)**: creation of a global schema

Data integration in the Multidatabase

We must build a system that:

- Supports access to different data sources
- “Knows” the contents of these data sources
- Integrates the different data sources by means of a unifying, global schema
- Receives queries expressed in the language of the global schema
- Distributes “rewritten” queries to the sources
- Combines the answers received from the sources to build the final answer

The general framework for Data Integration (recall)



(Alvaro Fernandez,
EDBT Summ.
School 2017)

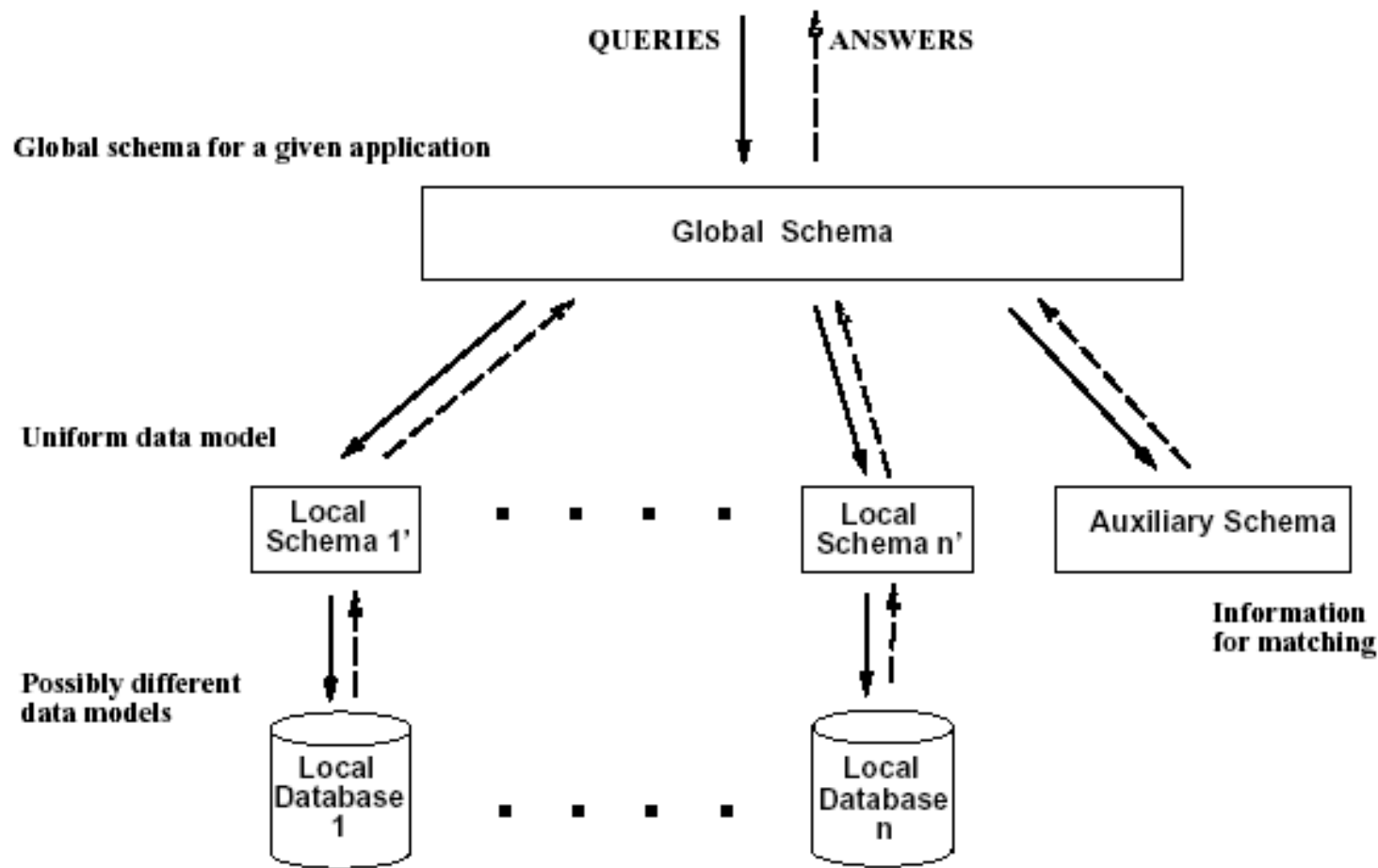
Many heterogeneities (recall):

- Same data model, different systems e.g. relational (Oracle, Sybase, DB2...) → technological heterogeneity
- Different data models, e.g. the very old *hierarchical or network* DBs (IMS, Codasyl...), **Relational**, Object Oriented, Graph-based.... → model heterogeneity
- Same data model, different query languages (SQL, Quel) → language heterogeneity
- Semi- or unstructured data (HTML, XML, multimedia...) → again model heterogeneity

A first case

- The data sources have the same data model
 - Adoption of a *global schema*
 - The global schema will provide a
 - Reconciled
 - Integrated
 - Virtual
- view of the data sources

Architecture with a uniform data model



Designing data integration in the Multidatabase (with the same data model)

1. Source schema identification (when present)
2. Source schema reverse engineering (data source conceptual schemata)
3. Conceptual schemata integration and restructuring: **related concept identification, conflict analysis and resolution**
4. Conceptual to logical translation (of the obtained **global conceptual schema**)
5. Mapping between the global logical schema and the single schemata (**logical view definition**)
6. After integration: **query-answering** through data views

Example (points 1, 2)

Two data sources: PoliRobots and UniRobots

PoliRobots:

REPORT (ID, dateTime, #ofFaults)

REPORTFAULT (reportID, faultID)

FAULT (ID, description, solution, responsibleOperatorID)

UniRobots:

MESSAGE (robotID, dateTime, errorCode) *//robotIDs are R001, R002, ..., R100*

ERROR (code, description, solution, personInCharge, urgency)

Additional note: PoliRobots has only one robot

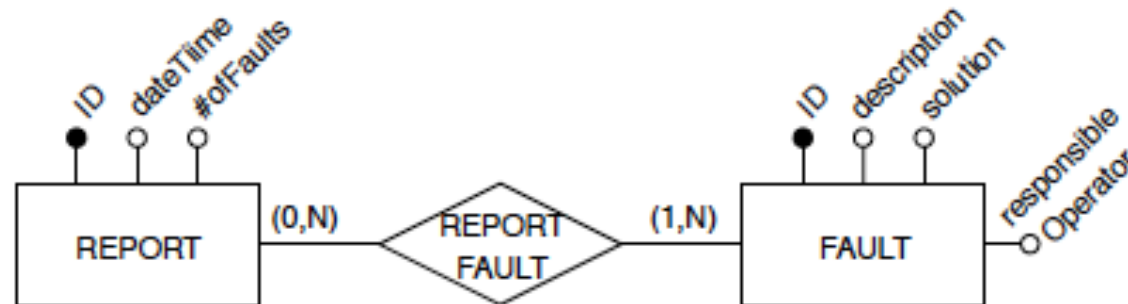
Example (points 1, 2)

PoliRobots:

REPORT (ID, dateTime, #ofFaults)

REPORTFAULT (reportID, faultID)

FAULT (ID, description, solution, responsibleOperatorID)

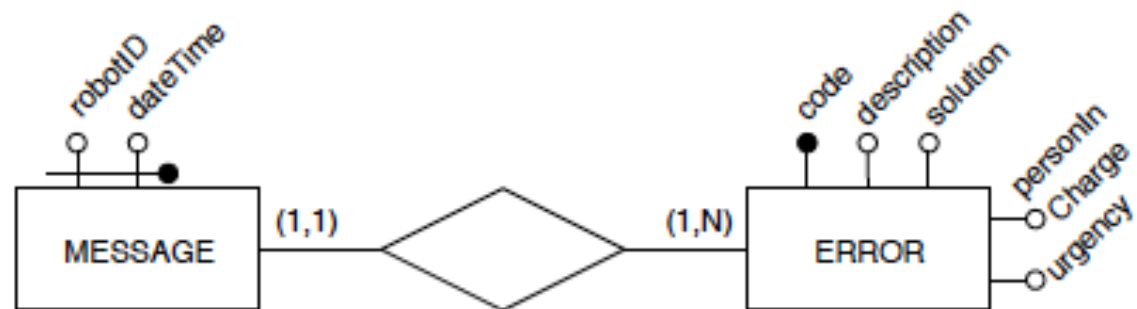


Example (points 1, 2)

UniRobots:

MESSAGE (robotID, dateTime, errorCode) //robotIDs are R001, R002, ..., R100

ERROR (code, description, solution, personInCharge, urgency)



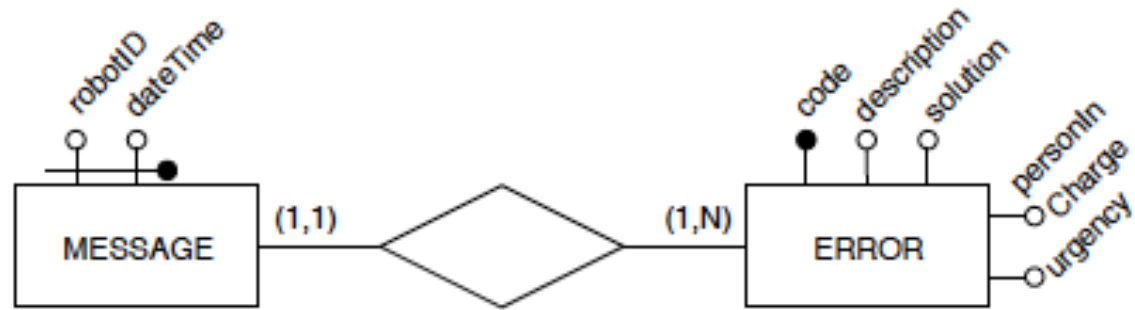
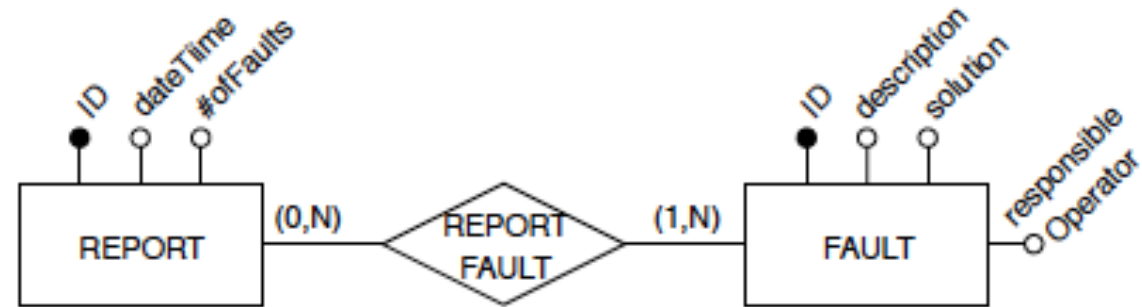
3. Conflict resolution and restructuring

- a. Related concept identification
- b. Conflict analysis and resolution
- c. Conceptual Schema integration

Related Concepts' identification

- Ex:
 - employee, clerk
 - exam, course
 - code, num
- Not too difficult if manual
- Much more difficult if automatic – we will see this later on

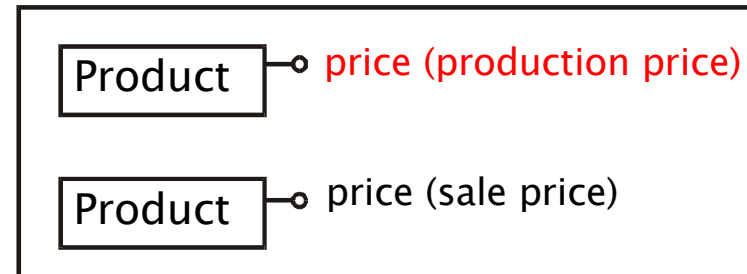
Related concept identification



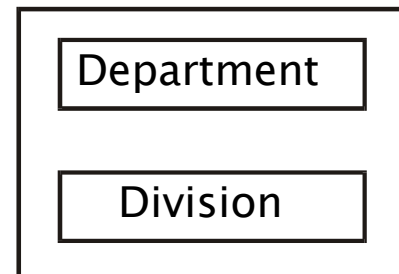
Conflict analysis

NAME CONFLICTS

- HOMONYMS



- SYNONYMS



Conflict analysis

TYPE CONFLICTS

- in a single attribute (e.g. NUMERIC, ALPHANUMERIC, ...)

e.g. the attribute “gender”:

- Male/Female
- M/F
- 0/1
- In Italy, it is implicit in the “codice fiscale” (SSN)

- in an entity type

different abstractions of the same real world concept produce different sets of properties (attributes)

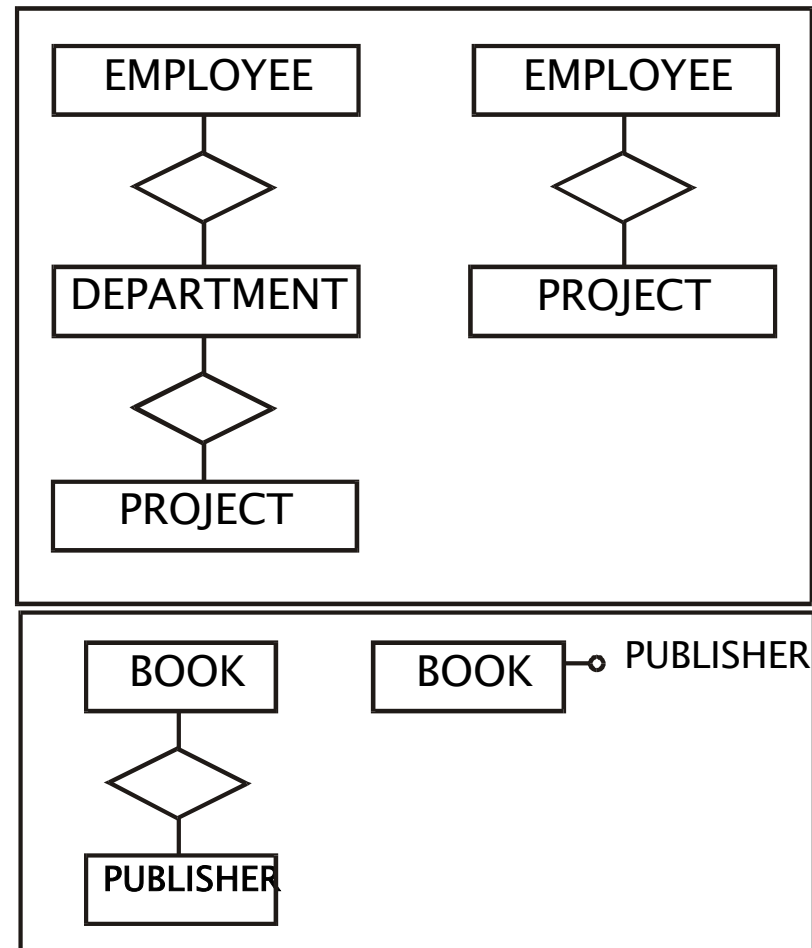
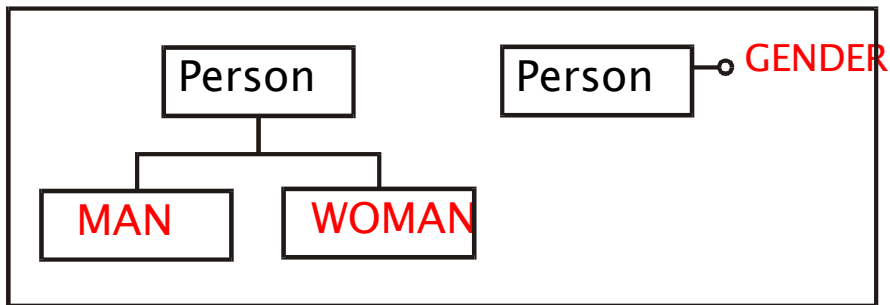
Conflict analysis

DATA SEMANTICS

- different currencies (euros, US dollars, etc.)
- different measure systems (kilos vs pounds, centigrades vs. Fahrenheit.)
- different granularities (grams, kilos, etc.)

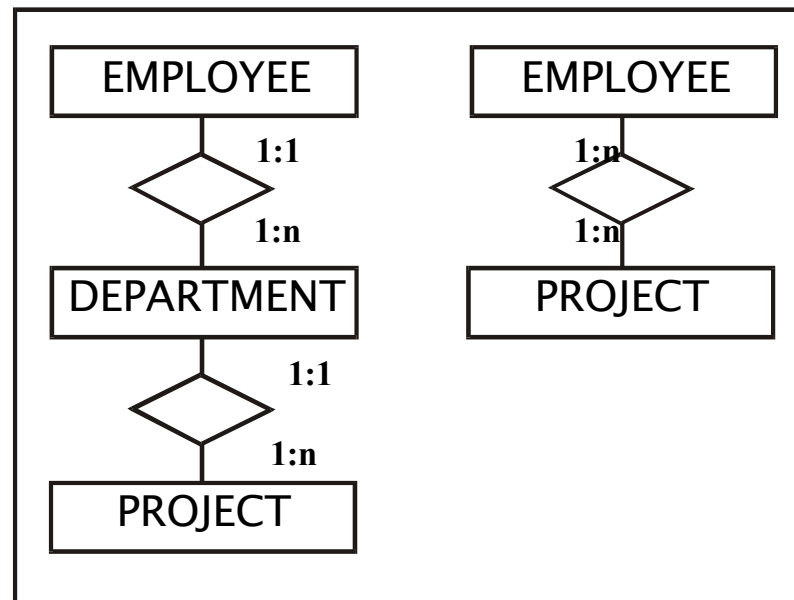
Conflict analysis

STRUCTURE CONFLICTS



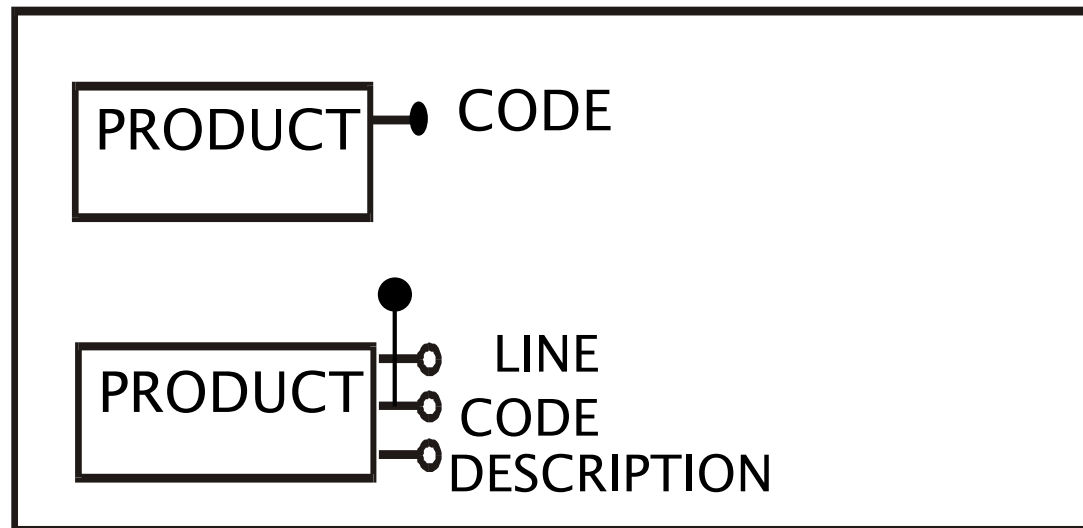
Conflict analysis

- DEPENDENCY (OR CARDINALITY) CONFLICTS

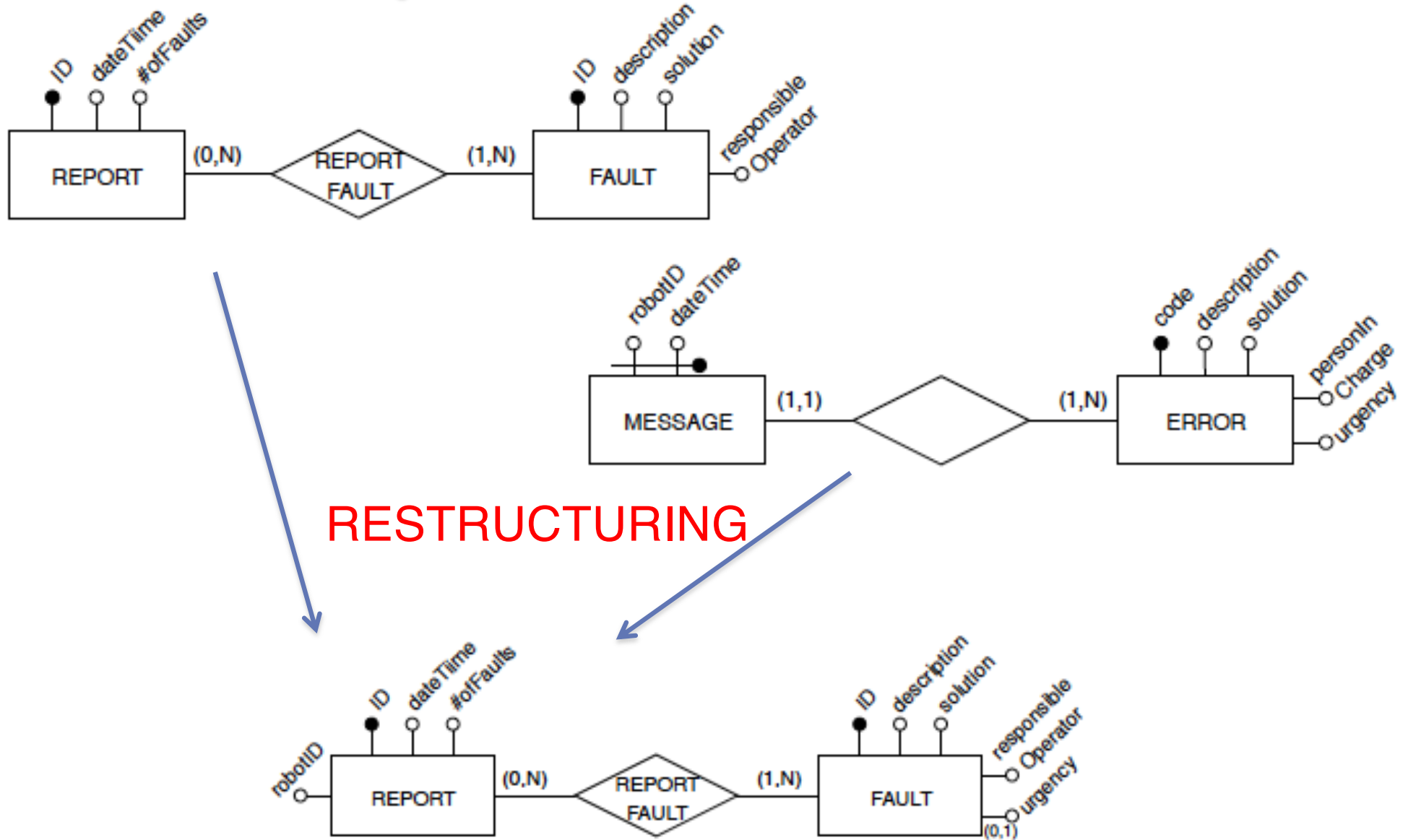


Conflict analysis

- KEY CONFLICTS



Example: UniPoliRobots



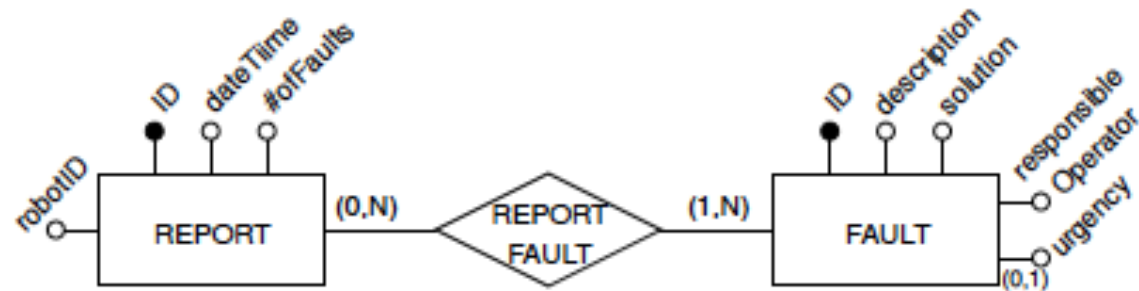
Designing data integration in the Multidatabase (with the same data model)

1. Source schema identification (when present)
2. Source schema reverse engineering (data source conceptual schemata)
3. Conceptual schemata integration and restructuring: conflict resolution and restructuring

NOW:

1. Conceptual to logical translation (of the obtained **global schema**)
5. Mapping between the global logical schema and the single schemata (**logical view definition**)
6. After integration: **query-answering** through data views

UniPoliRobots



REPORT(ID, RobotID, dateTime, #ofFaults)

FAULT(ID, description, solution, resp_operator, urgency*)

REPORT_FAULT(RID, FID)

```

CREATE VIEW UniPoliRobots.Report (ID, RobotID, dateTime, #ofFaults) AS
(
    SELECT KeyGenReport (ID, 'PoliRobots'), 'R101', dateTime, #of-
    Faults
    FROM PoliRobots.Report

    UNION

    SELECT KeyGenReport (robotID || dateTime, 'UniRobots'), robotID,
    dateTime, 1
    FROM UniRobots.Message
)
    
```

Additional note: R101 is the new ID for the PoliRobots robot